QUESTION 1: Toggle Light Bulbs

You are given an array bulbs of integers between 1 and 100.

There are 100 light bulbs numbered from 1 to 100. All of them are switched off initially.

For each element bulbs[i] in the array bulbs:
If the bulbs[i]th light bulb is currently off, switch it on.
Otherwise, switch it off.
Return the list of integers denoting the light bulbs that are on in the end, sorted in ascending order. If no bulb is on, return an empty list.

CORE CODING COMMITTEE
CODESPRINT
"RACE THE CLOCK. RULE THE CODE."

Example 1:

Input: bulbs = [10,30,20,10]

Output: [20,30]

Explanation:

The bulbs[0] = 10th light bulb is currently off. We switch it on.
The bulbs[1] = 30th light bulb is currently off. We switch it on.
The bulbs[2] = 20th light bulb is currently off. We switch it on.
The bulbs[3] = 10th light bulb is currently on. We switch it off.
In the end, the 20th and the 30th light bulbs are on.
Example 2:

Input: bulbs = [100,100]

Output: []

Explanation:

The bulbs[0] = 100th light bulb is currently off. We switch it on.
The bulbs[1] = 100th light bulb is currently on. We switch it off.
In the end, no light bulb is on.

Constraints:
1 <= bulbs.length <= 100
1 <= bulbs[i] <= 100

QUESTION 2: Check if Strings Can be Made Equal With Operations

You are given two strings s1 and s2, both of length n, consisting of lowercase English letters.

You can apply the following operation on any of the two strings any number of times:

Choose any two indices i and j such that i < j and the difference j - i is even, then swap the two characters at those indices in the string.
Return true if you can make the strings s1 and s2 equal, and false otherwise.

CORE CODING COMMITTEE 3

Example 1:

Input: s1 = "abcdba", s2 = "cabdab"
Output: true
Explanation: We can apply the following operations on s1:
- Choose the indices i = 0, j = 2. The resulting string is s1 = "cbadba".
- Choose the indices i = 2, j = 4. The resulting string is s1 = "cbbdaa".
- Choose the indices i = 1, j = 5. The resulting string is s1 = "cabdab" = s2.
Example 2:

CODESPRINT "RACE THE CLOCK, RULE THE CODE."

Input: s1 = "abe", s2 = "bea"
Output: false
Explanation: It is not possible to make the two strings equal.

Constraints:

n == s1.length == s2.length
1 <= n <= 105
s1 and s2 consist only of lowercase English letters.

QUESTION 3:LRU Cache

Design a data structure that follows the constraints of a Least Recently Used (LRU) cache.

Implement the LRUCache class:

LRUCache(int capacity) Initialize the LRU cache with positive size capacity.
int get(int key) Return the value of the key if the key exists, otherwise return -1.
void put(int key, int value) Update the value of the key if the key exists. Otherwise, add the key-value pair to the cache. If the number of keys exceeds the capacity from this operation, evict the least recently used key.
The functions get and put must each run in O(1) average time complexity.

Example 1:

Input
["LRUCache", "put", "put", "get", "put", "get", "put", "get", "get", "get"]
[[2], [1, 1], [2, 2], [1], [3, 3], [2], [4, 4], [1], [3], [4]]
Output
[null, null, null, 1, null, -1, null, -1, 3, 4]

Explanation
LRUCache lRUCache = new LRUCache(2);
lRUCache.put(1, 1); // cache is {1=1}
lRUCache.put(2, 2); // cache is {1=1, 2=2}
lRUCache.get(1);   // return 1
lRUCache.put(3, 3); // LRU key was 2, evicts key 2, cache is {1=1, 3=3}
lRUCache.get(2);   // returns -1 (not found)
lRUCache.put(4, 4); // LRU key was 1, evicts key 1, cache is {4=4, 3=3}
lRUCache.get(1);   // return -1 (not found)
lRUCache.get(3);   // return 3
lRUCache.get(4);   // return 4

CODESPRINT
"RACE THE CLOCK. RULE THE CODE."

QUESTION 4: Destroying Asteroids

You are given an integer mass, which represents the original mass of a planet. You are further given an integer array asteroids, where asteroids[i] is the mass of the ith asteroid.

You can arrange for the planet to collide with the asteroids in any arbitrary order. If the mass of the planet is greater than or equal to the mass of the asteroid, the asteroid is destroyed and the planet gains the mass of the asteroid. Otherwise, the planet is destroyed.

Return true if all asteroids can be destroyed. Otherwise, return false.

Example 1:

Input: mass = 10, asteroids = [3,9,19,5,21]
Output: true
Explanation: One way to order the asteroids is [9,19,5,3,21]:
- The planet collides with the asteroid with a mass of 9. New planet mass: 10 + 9 = 19
- The planet collides with the asteroid with a mass of 19. New planet mass: 19 + 19 = 38
- The planet collides with the asteroid with a mass of 5. New planet mass: 38 + 5 = 43
- The planet collides with the asteroid with a mass of 3. New planet mass: 43 + 3 = 46
- The planet collides with the asteroid with a mass of 21. New planet mass: 46 + 21 = 67
All asteroids are destroyed.
Example 2:

Input: mass = 5, asteroids = [4,9,23,4]
Output: false
Explanation:
The planet cannot ever gain enough mass to destroy the asteroid with a mass of 23.
After the planet destroys the other asteroids, it will have a mass of 5 + 4 + 9 + 4 = 22.
This is less than 23, so a collision would not destroy the last asteroid.

Constraints:

1 <= mass <= 105
1 <= asteroids.length <= 105
1 <= asteroids[i] <= 105

QUESTION 5: Number of Islands

Given an m x n 2D binary grid grid which represents a map of '1's (land) and '0's (water), return the number of islands.

An island is surrounded by water and is formed by connecting adjacent lands horizontally or vertically. You may assume all four edges of the grid are all surrounded by water.

Example 1:

Input: grid = [
  ["1","1","1","1","0"],
  ["1","1","0","1","0"],
  ["1","1","0","0","0"],
  ["0","0","0","0","0"]
]
Output: 1
Example 2:

Input: grid = [
  ["1","1","0","0","0"],
  ["1","1","0","0","0"],
  ["0","0","1","0","0"],
  ["0","0","0","1","1"]
]
Output: 3

Constraints:

m == grid.length
n == grid[i].length
1 <= m, n <= 300
grid[i][j] is '0' or '1'.

CORE CODING COMMITTEE 3
CODESPRINT
"RACE THE CLOCK. RULE THE CODE."

QUESTION 6: Course Schedule

There are n different online courses numbered from 1 to n. You are given an array courses where courses[i] = [durationi, lastDayi] indicate that the ith course should be taken continuously for durationi days and must be finished before or on lastDayi.

You will start on the 1st day and you cannot take two or more courses simultaneously.

Return the maximum number of courses that you can take.

Example 1:

Input: courses = [[100,200],[200,1300],[1000,1250],[2000,3200]]
Output: 3
Explanation:
There are totally 4 courses, but you can take 3 courses at most:
First, take the 1st course, it costs 100 days so you will finish it on the 100th day, and ready to take the next course on the 101st day.
Second, take the 3rd course, it costs 1000 days so you will finish it on the 1100th day, and ready to take the next course on the 1101st day.
Third, take the 2nd course, it costs 200 days so you will finish it on the 1300th day.
The 4th course cannot be taken now, since you will finish it on the 3300th day, which exceeds the closed date.
Example 2:

Input: courses = [[1,2]]
Output: 1
Example 3:

Input: courses = [[3,2],[4,3]]
Output: 0

Constraints:

1 <= courses.length <= 104
1 <= durationi, lastDayi <= 104

CODESPRINT
"RACE THE CLOCK. RULE THE CODE."

QUESTION 7: Prefix and Suffix Search

Design a special dictionary that searches the words in it by a prefix and a suffix.

Implement the WordFilter class:

WordFilter(string[] words) Initializes the object with the words in the dictionary.
f(string pref, string suff) Returns the index of the word in the dictionary, which has the prefix pref and the suffix suff. If there is more than one valid index, return the largest of them. If there is no such word in the dictionary, return -1.

CORE CODING COMMITTEE 3

Example 1:

Input
["WordFilter", "f"]
[[["apple"]], ["a", "e"]]
Output
[null, 0]
Explanation
WordFilter wordFilter = new WordFilter(["apple"]);
wordFilter.f("a", "e"); // return 0, because the word at index 0 has prefix = "a" and suffix = "e".

CODESPRINT
"RACE THE CLOCK. RULE THE CODE."

Constraints:

1 <= words.length <= 104
1 <= words[i].length <= 7
1 <= pref.length, suff.length <= 7
words[i], pref and suff consist of lowercase English letters only.
At most 104 calls will be made to the function f.

QUESTION 8: Stone Game

There are several stones arranged in a row, and each stone has an associated value which is an integer given in the array stoneValue.

In each round of the game, Alice divides the row into two non-empty rows (i.e. left row and right row), then Bob calculates the value of each row which is the sum of the values of all the stones in this row. Bob throws away the row which has the maximum value, and Alice's score increases by the value of the remaining row. If the value of the two rows are equal, Bob lets Alice decide which row will be thrown away. The next round starts with the remaining row.

The game ends when there is only one stone remaining. Alice's score is initially zero.

Return the maximum score that Alice can obtain.

Example 1:

Input: stoneValue = [6,2,3,4,5,5]
Output: 18
Explanation: In the first round, Alice divides the row to [6,2,3], [4,5,5]. The left row has the value 11 and the right row has value 14. Bob throws away the right row and Alice's score is now 11.
In the second round Alice divides the row to [6], [2,3]. This time Bob throws away the left row and Alice's score becomes 16 (11 + 5).
The last round Alice has only one choice to divide the row which is [2], [3]. Bob throws away the right row and Alice's score is now 18 (16 + 2). The game ends because only one stone is remaining in the row.
Example 2:

Input: stoneValue = [7,7,7,7,7,7,7]
Output: 28
Example 3:

Input: stoneValue = [4]
Output: 0

Constraints:

1 <= stoneValue.length <= 500
1 <= stoneValue[i] <= 106

QUESTION 9: LeaFall

You are given a tree∗ with n vertices. Over time, each vertex i(1≤i≤n) has a probability of pi/qi of falling. Determine the expected value of the number of unordered pairs of distinct vertices that become leaves in the resulting forest, modulo 998 244 353
.
Note that when vertex v falls, it is removed along with all edges connected to it. However, adjacent vertices remain unaffected by the fall of v

---

A tree is a connected graph without cycles.
An unordered pair is a collection of two elements where the order in which the elements appear does not matter. For example, the unordered pair (1,2) is considered the same as (2,1)
A leaf is a vertex that is connected to exactly one edge.
A forest is a graph without cycles

Input
Each test contains multiple test cases. The first line contains the number of test cases t(1≤t≤104). The description of the test cases follows.

The first line of each test case contains a single integer n(1≤n≤105).

The i-th line of the following n lines contains two integers pi and qi (1≤pi<qi<998 244 353).

Each of the following n−1 lines contains two integers u and v (1≤u,v≤n, u≠v) — the indices of the vertices connected by an edge.

It is guaranteed that the given edges form a tree.

It is guaranteed that the sum of n over all test cases does not exceed 105.

Output

For each test case, output a single integer — the expected value of the number of unordered pairs of distinct vertices that become leaves in the resulting forest modulo 998244353.

Formally, let M=998244353.It can be shown that the exact answer can be expressed as an irreducible fraction pq, where p and q are integers and $q \not\equiv 0 \pmod{M}$.
Output the integer equal to $p \cdot q^{-1} \bmod M$. In other words, output such an integer x that $0 \le x < M$
and $x \cdot q \equiv p \pmod{M}$.

Examples

InputCopy
5
1
1 2
3
1 2
1 2
1 2
1 2
2 3
3
1 3
1 5
1 3
1 2
2 3
1
998244351 998244352
6
10 17
7 13
6 11
2 10
10 19
5 13

CORE CODING COMMITTEE
CODESPRINT
"RACE THE CLOCK. RULE THE CODE."

4 3
3 6
1 4
3 5
3 2

OutputCopy
0
623902721
244015287
0
799215919

InputCopy
1
10
282508078 551568452
894311255 989959022
893400641 913415297
460925436 801908985
94460427 171411253
997964895 998217862
770266391 885105593
591419316 976424827
606447024 863339056
940224886 994244553
9 5
9 6
9 8
8 7
3 6
1 5
7 4
8 10
4 2

OutputCopy

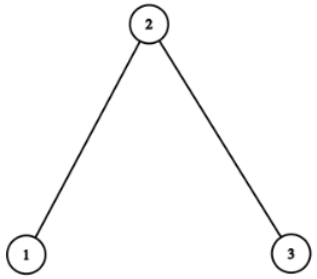
CORE CODING COMMITTEE
CODESPRINT
"RACE THE CLOCK. RULE THE CODE."

486341067

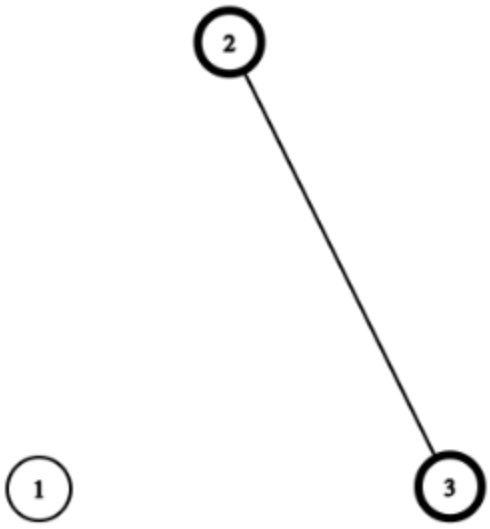
Note

In the first test case, only one vertex is in the tree, which is not a leaf, so the answer is 0 .

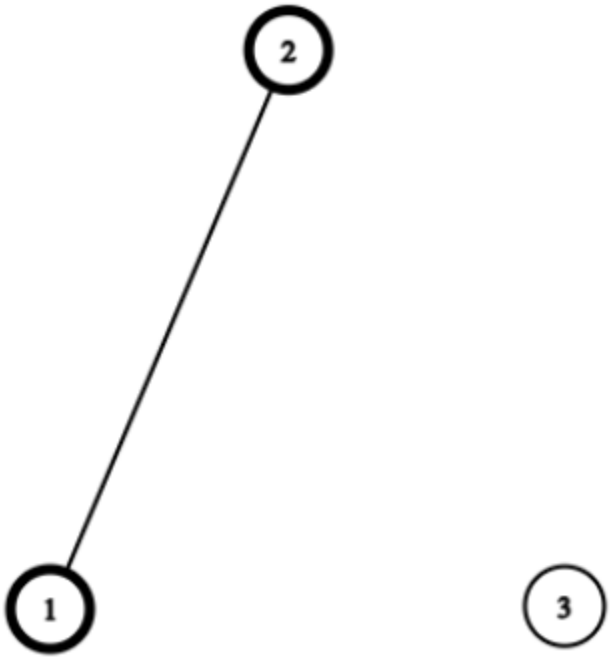
In the second test case, the tree is shown below.



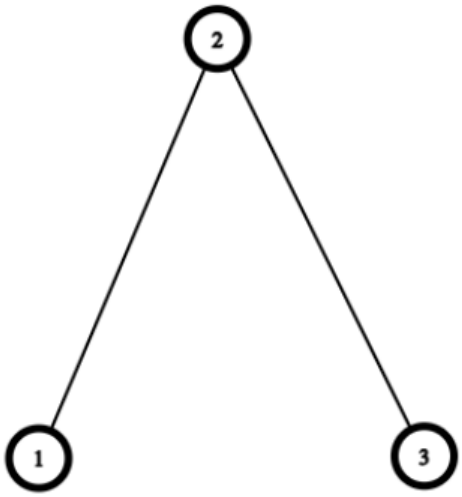
Vertices that have not fallen are denoted in bold. Let us examine the following three cases:



We arrive at this configuration with a probability of (1/2)3, where the only unordered pair of distinct leaf vertices is (2,3)



.We arrive at this configuration with a probability of (1/2)3, where the only unordered pair of distinct leaf vertices is (2,1)



.We arrive at this configuration with a probability of (1/2)3, where the only unordered pair of distinct leaf vertices is (1,3)

.

All remaining cases contain no unordered pairs of distinct leaf vertices. Hence, the answer is 1+1+18=38, which is equal to 623 902 721 modulo 998244353

QUESTION 10: Cracking the Safe

There is a safe protected by a password. The password is a sequence of n digits where each digit can be in the range [0, k - 1].

The safe has a peculiar way of checking the password. When you enter in a sequence, it checks the most recent n digits that were entered each time you type a digit.

For example, the correct password is "345" and you enter in "012345":
After typing 0, the most recent 3 digits is "0", which is incorrect.
After typing 1, the most recent 3 digits is "01", which is incorrect.
After typing 2, the most recent 3 digits is "012", which is incorrect.
After typing 3, the most recent 3 digits is "123", which is incorrect.
After typing 4, the most recent 3 digits is "234", which is incorrect.
After typing 5, the most recent 3 digits is "345", which is correct and the safe unlocks.
Return any string of minimum length that will unlock the safe at some point of entering it.

CODESPRINT
RACE THE CLOCK. RULE THE CODE.

Example 1:
Input: n = 1, k = 2
Output: "10"
Explanation: The password is a single digit, so enter each digit. "01" would also unlock the safe.

Example 2:
Input: n = 2, k = 2
Output: "01100"
Explanation: For each possible password:
- "00" is typed in starting from the 4th digit.
- "01" is typed in starting from the 1st digit.
- "10" is typed in starting from the 3rd digit.
- "11" is typed in starting from the 2nd digit.
Thus "01100" will unlock the safe. "10011", and "11001" would also unlock the safe.

Constraints:
1 <= n <= 4

1 <= k <= 10
1 <= kn <= 4096

CORE CODING COMMITTEE
3
CODESPRINT
"RACE THE CLOCK. RULE THE CODE."